

NumPy for Signals & Linear Systems

A cheat sheet for Python signal problems: time axes, transformations, energy/power, even-odd, interpolation

1. Mindset & Imports

Think in **whole arrays**, not loops. Any operation you'd write as a C++ `for` loop over samples usually becomes one array expression.

```
import numpy as np
import matplotlib.pyplot as plt
```

If you're writing a loop over every sample, pause — there's almost always a vectorised one-liner.

2. Building the Time Axis

linspace evenly spaced, **both ends included**. Use for continuous $x(t)$.

```
# 4001 points from -4 to 4
t = np.linspace(-4.0, 4.0, 4001)
# spacing dt = 8/4000 = 0.002
```

arange like a for-loop, **stop excluded**. Use for discrete $x[n]$ (integer index).

```
n = np.arange(-20, 21) # -20..20
# NOTE: 21 to INCLUDE 20
np.arange(0, 10, 2) # 0 2 4 6 8
```

Prefilled arrays

```
np.zeros(5) # 0 0 0 0 0
np.ones(5) # 1 1 1 1 1
np.zeros_like(t) # zeros, shape of t
np.full(5, 3.0) # 3 3 3 3 3
```

3. Element-wise Math & Broadcasting

Every op applies to each element. A scalar “broadcasts” to all.

```
x + 10 # add to every element
x * 2 # scale amplitude by 2
x ** 2 # square (for energy!)
x + y # element-wise add arrays
-x # flip sign (odd part)
```

Vectorised math functions:

```
np.sin(t) np.cos(t) np.exp(t)
np.abs(x) np.sqrt(x) np.sign(x)
np.pi # 3.14159...
```

Sinusoid pattern memorise it:

```
# continuous: freq f Hz
x = A*np.sin(2*np.pi*f*t + phi)
# discrete: x[n]=A cos(W0 n + phi)
```

```
x = A*np.cos(0*omega*n + phi)
```

4. Indexing & Slicing

```
x[0] # first
x[-1] # last
x[1:4] # idx 1,2,3 (stop excl.)
x[:3] # first three
x[2:] # from idx 2 to end
```

Reversal — time reversal $x(-t)$

```
x[::-1] # whole array reversed
```

On a *symmetric* axis (e.g. $-4 \dots 4$), the samples of $x(-t)$ are just the samples of $x(t)$ read backwards: `x[::-1]`.

5. Boolean Masks (piecewise signals)

A comparison on an array returns a True/False array; use it to read or assign a region.

```
mask = np.abs(t) <= 1.0
tri = np.zeros_like(t)
# fill ONLY where mask is True
tri[mask] = 1.0 - np.abs(t[mask])
```

Combine masks with `&`, `|`, `~` and **parentheses**:

```
m = (t >= 0) & (t <= 2) # AND
m = (t < -1) | (t > 1) # OR
# NEVER use 'and'/'or' on arrays
```

Building blocks for signals:

```
# rectangular pulse (window)
rect = np.zeros_like(t)
rect[(t>=-1)&(t<=1)] = 1.0
# ramp on a region
ramp = np.zeros_like(t)
m = np.abs(t) <= 1.0
ramp[m] = t[m]
# shifted triangle center a
u = t - a
m = np.abs(u) <= 1.0
tri = np.zeros_like(t)
tri[m] = 1.0 - np.abs(u[m])
```

`np.where` is a compact alternative:

```
x = np.where(t>=0, 1.0, 0.0) # step
```

6. Reductions: Energy & Power

```
np.sum(x) np.mean(x)
np.max(x) np.min(x)
np.cumsum(x) # running sum
```

From the lecture (discrete):

$$E = \sum |x[n]|^2 \quad P = \frac{1}{N} \sum |x[n]|^2$$

```
E = np.sum(np.abs(x)**2)
P = np.mean(np.abs(x)**2)
```

Continuous (approx. integral, dt spacing):

$$E = \int |x(t)|^2 dt$$

```
dt = t[1]-t[0]
E = np.sum(np.abs(x)**2)*dt
# or: np.trapz(np.abs(x)**2, t)
```

MSE (compare two signals)

```
mse = np.mean((a - b)**2)
```

7. Transformations Recap

Shift $y(t) = x(t - t_0)$: recompute the formula with shifted argument (no boundary issues).

```
def shift(n, A, W0, phi, n0):
    return A*np.cos(W0*(n-n0)+phi)
```

Array-index shift (fill vacated slots with 0):

```
def shift_arr(x, n0):
    y = np.zeros_like(x)
    if n0 >= 0:
        y[n0:] = x[:len(x)-n0]
    else:
        y[:n0] = x[-n0:]
    return y
```

Reversal $x(-t)$: `x[::-1]` (symmetric grid) or interpolate at $-t$.

Scaling $y(t) = x(t/k)$ needs values between samples $\Rightarrow$ **interpolate**.

8. Interpolation (sub-scaling)

Read x at query times; average nearest neighbours when between samples. $y(1) = \frac{1}{2}(x(0) + x(1))$.

```
def interp(t, x, tq):
    dt = t[1]-t[0]; t0 = t[0]
    y = np.zeros_like(tq)
    for i, q in enumerate(tq):
        p = (q - t0)/dt # frac index
        if p < 0 or p > len(t)-1:
            continue # out of range
        lo = int(np.floor(p))
        hi = int(np.ceil(p))
        if lo == hi:
            y[i] = x[lo]
        else:
            y[i] = 0.5*(x[lo]+x[hi])
    return y

def time_scale(t, x, k):
    return interp(t, x, t/k)
```

Built-in shortcut (linear interp):

```
y = np.interp(t/k, t, x)
```

Float trap: `(q-t0)/dt` can give `2000.0000002` instead of `2000` .
Snap it: `if np.isclose(p,round(p)): p=round(p)` so exact hits aren't averaged.

9. Even–Odd Decomposition

$$x_e = \frac{1}{2}[x(t) + x(-t)], \quad x_o = \frac{1}{2}[x(t) - x(-t)], \quad x = x_e + x_o.$$

```
def time_reverse(x):
    return x[::-1]

def even_odd(x):
    xr = time_reverse(x)
    xe = 0.5*(x + xr)
    xo = 0.5*(x - xr)
```

```
return xe, xo
```

Check: `xe[::-1]==xe` (even) and `xo[::-1]==-xo` (odd). Works only if the time axis is symmetric about 0.

10. Shift ↔ Phase (discrete sinusoid)

$$\cos(\Omega_0(n-n_0) + \phi) = \cos(\Omega_0 n + \phi - \Omega_0 n_0).$$

- A time shift n_0 *always* equals a phase shift $\phi_0 = -\Omega_0 n_0$.
- A phase shift only equals an *integer* time shift if $\phi_0/(-\Omega_0)$ is an integer — usually it isn't.

```
phi0_equiv = -Omega0*n0 # exact
# search best integer shift:
best = min(range(-12,13),
            key=lambda k: np.mean(
                (shift(n,A,W0,phi,k)-x_phase)**2))
```

11. Plotting (Matplotlib)

Continuous

```
plt.figure()
plt.plot(t, x, label="x(t)")
plt.plot(t, y, label="y(t)")
plt.title("..."); plt.xlabel("t")
plt.ylabel("Amplitude")
plt.grid(True); plt.legend()
plt.show()
```

Discrete — stems

```
fig, ax = plt.subplots()
ml, sl, bl = ax.stem(n, x,
```

```
label="x[n]")
bl.set_visible(False)
ax.grid(True, alpha=0.3)
ax.set_xlabel("n"); ax.legend()
```

Subplots & limits

```
fig, (a1,a2) = plt.subplots(1,2,
                             figsize=(9,4))
a1.plot(t,x); a2.plot(t,y)
plt.xlim(-4,4); plt.ylim(-2,2)
plt.tight_layout()
```

12. Gotchas & Quick Fixes

- `arange(a,b)` excludes `b` ; add 1 to include it.
- Use `&` | `~` (not `and` or `not`) on arrays; wrap comparisons in `()` .
- Integer array $\div$ int can surprise — make axes `float` (`-4.0`).
- `x[::-1]` = reversal only on a **symmetric** axis about 0.
- Copy vs view: `y = x` shares memory; use `y = x.copy()` to be safe.
- Compare floats with `np.isclose` , never `==` .
- Shapes must match for `a-b` ; check `x.shape` , `len(x)` .

Inspecting arrays

```
x.shape    x.dtype    len(x)
x.min()    x.max()    x.mean()
np.allclose(a, b) # equal-ish?
```